
NumPy for Machine Learning

A complete set of study notes — arrays, shapes, indexing, broadcasting, matrix multiplication, and the NumPy foundation behind linear regression, logistic regression, and neural networks.

This guide focuses less on random NumPy functions and more on the concepts that matter once you reach linear regression, logistic regression, gradient descent, and deep learning. The standard import used throughout:

IMPORT

```
import numpy as np
```

np is just an alias — instead of writing `numpy.array(...)`, we write `np.array(...)`.

1. Creating NumPy arrays

The most basic syntax is `np.array([values])`.

CODE

```
import numpy as np

x = np.array([1, 2, 3, 4, 5])

print(x)
```

OUTPUT

```
[1 2 3 4 5]
```

For a matrix:

CODE

```
x = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
```

2. Dimensions: 1D, 2D, 3D...

This is extremely important in ML.

1D

CODE

```
x = np.array([1, 2, 3])
```

Shape: (3,)

2D

CODE

```
X = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
```

Shape: (2, 3) — meaning 2 rows, 3 columns.

3D

CODE

```
X = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])
```

Shape: (2, 2, 2) — think of it as 2 matrices, each with 2 rows, each row with 2 values.

3. .shape — one of the MOST important things

CODE

```
X.shape
```

CODE

```
X = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

```
print(X.shape)
```

OUTPUT

```
(2, 3)
```

This means 2 rows and 3 columns.

In ML

If `X.shape` returns `(1000, 20)`, that usually means:

- **1000** → training examples
- **20** → features

Understanding shape is essential.

4. `.ndim`

This tells you the number of dimensions.

CODE

```
x = np.array([1, 2, 3])
print(x.ndim)
```

OUTPUT

```
1
```

CODE

```
X = np.array([
    [1, 2],
    [3, 4]
])
print(X.ndim)
```

OUTPUT

```
2
```

ndim → how many dimensions? **shape** → how large is each dimension?

5. `.size`

This tells you the total number of elements.

CODE

```
X = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(X.size)
```

OUTPUT

```
6
```

6. Indexing

Just like Python lists:

CODE

```
x = np.array([10, 20, 30, 40])
print(x[0])
```

OUTPUT

```
10
```

CODE

```
print(x[2])
```

OUTPUT

```
30
```

7. 2D indexing

This is very important.

CODE

```
x = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])
```

`x[0, 0]` means row 0, column 0 → result 10.

`x[1, 2]` means row 1, column 2 → result 60.

Remember: NumPy uses zero-based indexing, just like normal Python.

8. Selecting entire rows

CODE

```
x[0, :]
```

Means: take row 0, and take all columns.

OUTPUT

```
[10 20 30]
```

You can also write `x[0]` for the same result.

9. Selecting entire columns

This is particularly useful in ML.

CODE

```
X[:, 0]
```

Means: take all rows, but only column 0.

OUTPUT

```
[10 40 70]
```

Similarly, `X[:, 1]` gives `[20 50 80]`. This is how you might select a particular feature from a dataset.

10. Slicing

CODE

```
x = np.array([10, 20, 30, 40, 50])  
x[1:4]
```

OUTPUT

```
[20 30 40]
```

Same basic slicing rule as Python: the stop position is excluded.

11. Creating arrays of zeros and ones

Very common in ML:

CODE

```
np.zeros(5)
```

OUTPUT

```
[0. 0. 0. 0. 0.]
```

CODE

```
np.zeros((3, 4)) # a 3x4 matrix of zeros  
np.ones((2, 3))
```

OUTPUT

```
[[1. 1. 1.]  
 [1. 1. 1.]
```

These become useful for initializing data, parameters, masks, etc.

12. arange()

CODE

```
np.arange(start, stop, step)
```

CODE

```
np.arange(0, 10, 2)
```

OUTPUT

```
[0 2 4 6 8]
```

Notice that 10 isn't included.

13. linspace()

CODE

```
np.linspace(start, stop, number_of_values)
```

CODE

```
np.linspace(0, 1, 5)
```

OUTPUT

```
[0. 0.25 0.5 0.75 1. ]
```

Particularly useful when you need evenly spaced numerical values.

14. Element-wise operations

This is where NumPy becomes extremely powerful.

CODE

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b
```

OUTPUT

```
[5 7 9]
```

Because NumPy performs the operation element-by-element. Similarly:

CODE

```
a * b
```

OUTPUT

```
[4 10 18]
```

Important: * does element-by-element multiplication, **not** matrix multiplication.

15. Matrix multiplication — VERY important

One of the most important NumPy concepts for ML. Use A @ B.

CODE

```
A = np.array([
    [1, 2],
    [3, 4]
])

B = np.array([
    [5, 6],
    [7, 8]
])

C = A @ B
```

Each output element comes from multiplying a row of A by a column of B and adding the results.

16. * vs @

This distinction is critical.

Element-wise multiplication

CODE

```
A * B
```

Multiplies matching positions together.

Matrix multiplication

CODE

```
A @ B
```

Performs true linear-algebra matrix multiplication. In ML, this difference matters enormously.

17. Dot product

For vectors:

CODE

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.dot(a, b)
```

You can also use `a @ b` for vectors — `np.dot(a, b)` and `a @ b` represent the same dot product in this situation.

18. Why dot products matter in ML

This is where NumPy starts connecting directly to the models you're learning. A model calculates a weighted sum of inputs — in NumPy:

CODE

```
z = w @ x
```

This is the foundation of many ML calculations.

19. Transpose .T

CODE

```
x = np.array([
    [1, 2, 3]
])
```

This has shape `(1, 3)`. Transpose:

CODE

```
x.T
```

gives shape `(3, 1)` — a row becomes a column. You'll see transpose constantly in ML mathematics.

20. Broadcasting — one of the “complex” concepts

This is something you really should understand.

CODE

```
x = np.array([1, 2, 3])  
x + 10
```

OUTPUT

```
[11 12 13]
```

There isn't actually a 10 stored three times — NumPy conceptually broadcasts the scalar 10 against each element of [1, 2, 3]. That's broadcasting.

21. Broadcasting with a matrix

CODE

```
X = np.array([  
    [1, 2, 3],  
    [4, 5, 6]  
)  
  
X + 10
```

OUTPUT

```
[[11 12 13]  
 [14 15 16]]
```

The scalar 10 is applied to every element. But broadcasting gets more interesting here:

CODE

```
X + np.array([10, 20, 30])
```

NumPy effectively applies [10 20 30] to every row:

OUTPUT

```
[11 22 33]  
[14 25 36]
```

22. Why broadcasting matters in ML

Suppose you have 1000 students and 5 features:

CODE

```
X.shape = (1000, 5)
```

Maybe you want to subtract the mean of each feature. The mean might have shape (5,). Then:

CODE

```
X - mean
```

automatically subtracts the corresponding feature mean from every row. This is one of the reasons NumPy can make ML code incredibly compact.

23. Reshaping

CODE

```
x = np.array([1, 2, 3, 4, 5, 6]) # shape (6,)  
  
x.reshape(2, 3)
```

OUTPUT

```
[[1 2 3]  
 [4 5 6]] # shape (2, 3)
```

CODE

```
x.reshape(3, 2)
```

OUTPUT

```
[[1 2]  
 [3 4]  
 [5 6]]
```

24. The number of elements must remain the same

With 6 elements you can reshape into (2, 3) or (3, 2), but not (4, 3) — that would need 12 elements, and you only have 6.

25. -1 in reshape

This looks strange at first:

CODE

```
x.reshape(2, -1)
```

NumPy says: “I know the first dimension is 2. You calculate the other dimension for me.” Since there are 6 elements, NumPy determines (2, 3). This pattern is extremely common.

26. Flattening

CODE

```
X = np.array([
    [1, 2],
    [3, 4]
])

X.flatten()
```

OUTPUT

```
[1 2 3 4]
```

This turns a multidimensional array into a 1D array.

27. Aggregation functions

NumPy can calculate statistics across arrays:

CODE

```
x = np.array([10, 20, 30, 40])
```

Function	Meaning	Result
<code>np.sum(x)</code>	Sum	100
<code>np.mean(x)</code>	Mean	25
<code>np.max(x)</code>	Maximum	40
<code>np.min(x)</code>	Minimum	10
<code>np.std(x)</code>	Standard deviation	—

These become useful when preprocessing data.

28. axis — VERY important and initially confusing

This is one of the concepts you should spend time understanding.

CODE

```
x = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

np.sum(X)
```

OUTPUT

```
21 # everything is summed
```

CODE

```
np.sum(X, axis=0)
```

OUTPUT

```
[5 7 9] # sums down the rows, column by column
```

CODE

```
np.sum(X, axis=1)
```

OUTPUT

```
[6 15] # sums across each row
```

A useful mental model:

- **axis=0** → collapse rows → results for each column
- **axis=1** → collapse columns → results for each row

This becomes very important when preprocessing ML datasets.

29. Boolean masking

CODE

```
x = np.array([10, 25, 5, 40, 15])  
x > 20
```

OUTPUT

```
[False True False True False]
```

CODE

```
x[x > 20]
```

OUTPUT

```
[25 40]
```

So you're saying: "Give me only the values where the condition is true." This is extremely useful for data manipulation.

30. np.where()

CODE

```
np.where(x > 20, 1, 0)
```

Means: "If $x > 20$, give me 1; otherwise give me 0."

OUTPUT

```
[0 1 0 1 0]
```

This kind of operation becomes useful when creating labels or transforming data.

31. Random numbers

Machine learning uses randomness in many places.

CODE

```
np.random.rand(3) # uniform [0, 1)
np.random.randn(3) # standard normal
np.random.randint(1, 10, size=5)
```

OUTPUT

```
[3 8 1 7 4] # the exact result varies
```

32. Random seed

If you want reproducible random results:

CODE

```
np.random.seed(42)
np.random.rand(3)
```

This will produce the same sequence each time you start from that seed — very useful when debugging or comparing ML experiments.

33. Mathematical functions

Function	Meaning
<code>np.sqrt(x)</code>	Square root
<code>np.exp(x)</code>	Exponential
<code>np.log(x)</code>	Natural logarithm
<code>np.sin(x)</code>	Sine
<code>np.abs(x)</code>	Absolute value

`np.exp()` is particularly important for the sigmoid function:

CODE

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

34. Now let's connect NumPy directly to ML

CODE

```
X = np.array([
    [2, 3],
    [4, 5],
    [6, 7]
]) # 3 examples, 2 features

X.shape # (3, 2)
```

CODE

```
w = np.array([0.5, 0.8]) # weights

z = X @ w
```

OUTPUT

```
z ≈ [3.4 6.0 8.6]
```

Then add a bias and apply sigmoid:

CODE

```
b = -2
z = X @ w + b

predictions = 1 / (1 + np.exp(-z))
```

You have just implemented a tiny piece of **logistic regression** using NumPy.

35. This is why NumPy matters so much

ML equations that look like scary mathematics turn into a few lines of code:

CODE

```
Z = X @ w + b
A = sigmoid(Z)
```

That's the bridge between ML mathematics and actual Python code.

36. The NumPy concepts you should master

Don't try to memorize 500 NumPy functions. For your AI/ML path, prioritize these:

Beginner

<code>np.array()</code>	<code>np.zeros()</code>	<code>np.ones()</code>
<code>np.arange()</code>	<code>np.linspace()</code>	<code>.shape</code>
<code>.ndim</code>	<code>.size</code>	indexing
slicing		

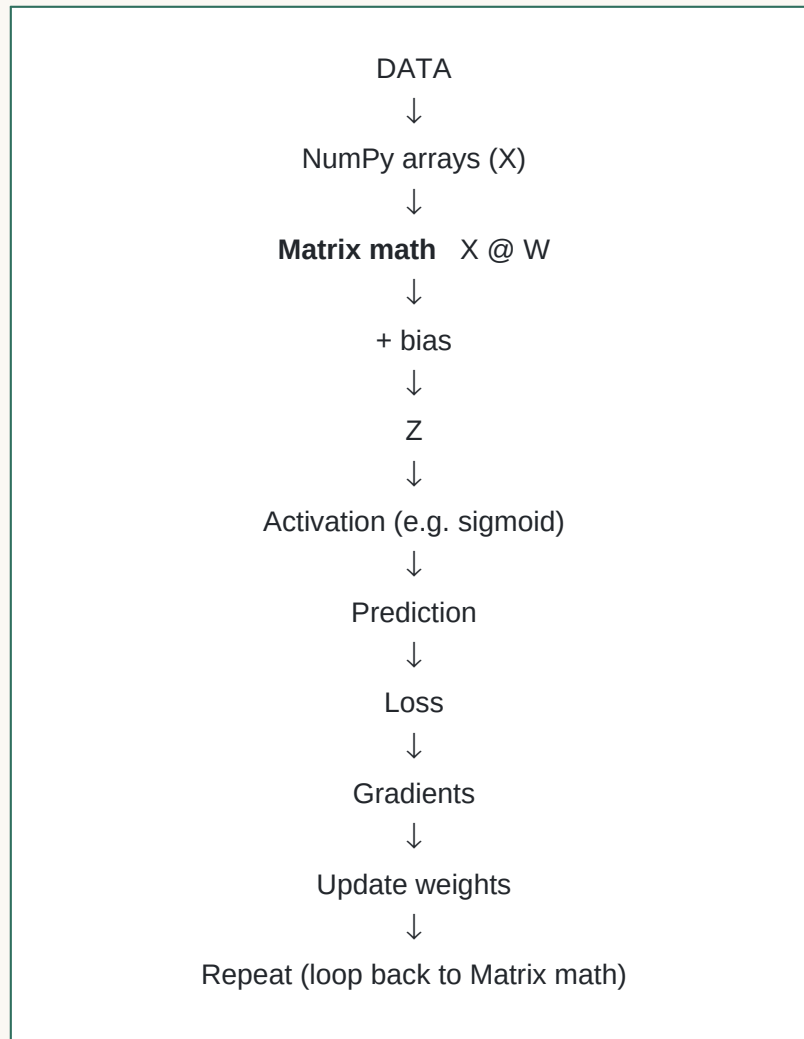
Intermediate

<code>.reshape()</code>	<code>.flatten()</code>	<code>.sum()</code>
<code>.mean()</code>	<code>.min()</code>	<code>.max()</code>
<code>.std()</code>	axis	<code>np.where()</code>
boolean indexing		

Very important for ML — master these especially

*	@	<code>np.dot()</code>
<code>.T</code>	broadcasting	reshape
axis	vectorization	<code>np.exp()</code>
<code>np.log()</code>	<code>np.random...</code>	

The big picture



NumPy is the mathematical machinery that lets you manipulate the arrays involved in all of this. If you understand **shape** → **indexing** → **slicing** → **broadcasting** → **axis** → **reshape** → **dot product** → **@** **matrix multiplication** → **transpose** → **vectorization**, you'll have the NumPy foundation you need before going deeply into ML.